

Chapter 4: Implementation

BusTrack — Real-Time Public Transport Tracking and Crowd Density
Prediction System

Yohannes Desalegn

June 10, 2026

Chapter 4: Implementation

This chapter focuses on the actual realization of the proposed BusTrack system. While Chapter 3 discusses the design and methodology, Chapter 4 explains how the system was physically and logically implemented, integrated, configured, and debugged. The implementation follows the architectural decisions made in the design phase, translating the three-tier architecture (Physical Collection, Cloud Processing, User Presentation) into working software, hardware, and machine learning components.

4.1 Overview

The BusTrack system was implemented using a bottom-up approach, beginning with the hardware firmware, followed by the backend API, then the frontend dashboards and mobile application, and finally the machine learning pipelines. This approach ensured that each layer was fully functional before integrating it with the layers above, reducing debugging complexity and enabling parallel development across subsystems.

The implementation spans four major subsystems:

- **Firmware** — ESP32-CAM on each bus captures GPS coordinates and camera images
- **Backend** — FastAPI server processes telemetry, runs CV/ML algorithms, serves data via REST and WebSocket
- **Dashboards** — Next.js admin and bus driver interfaces for fleet management and ride operations
- **Mobile App** — Flutter passenger application for real-time bus tracking and crowd density information

The technology stack was selected to balance performance, developer productivity, and deployment flexibility:

Component	Technology	Purpose
Backend	Python 3.11, FastAPI, SQLAlchemy async	API server, data processing
Database	PostgreSQL 15, PostGIS	Geospatial data storage
Cache	Redis 7	Real-time caching, pub/sub messaging
Admin Dashboard	Next.js 14, TypeScript, Tailwind CSS, shadcn/ui	Fleet management interface
Driver Dashboard	Next.js 14, TypeScript, Tailwind CSS	Bus driver ride interface
Mobile App	Flutter 3.x, Dart 3, Riverpod, GoRouter	Passenger tracking application
ML — Crowd	YOLOv8-nano, OpenCV	Crowd density detection
ML — ETA	RandomForest (scikit-learn)	ETA prediction correction
Firmware	Arduino/C++, ESP32-CAM, NEO-6M	On-bus data collection

Table 4.1: Technology stack summary for the BusTrack system.

4.2 Hardware Implementation

The hardware subsystem consists of an ESP32-CAM module paired with a NEO-6M GPS receiver, mounted inside each bus. This section describes the component selection rationale, physical assembly, wiring, and firmware implementation.

4.2.1 ESP32-CAM and NEO-6M GPS Module Integration

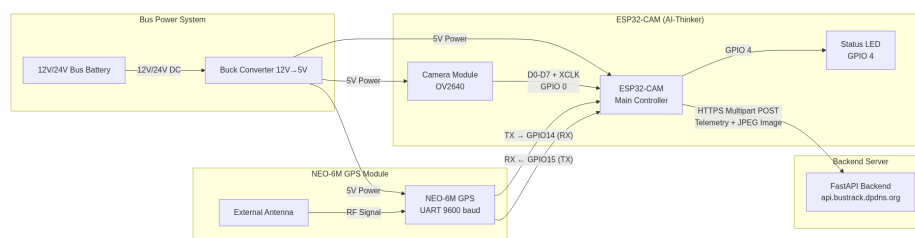
The ESP32-CAM (AI-Thinker variant) was selected as the primary onboard computing unit for several practical reasons:

- Dual-core Xtensa LX6 processor at 240 MHz
- 520 KB SRAM + up to 4 MB external PSRAM for JPEG buffering
- Built-in Wi-Fi (802.11 b/g/n) eliminates need for separate wireless module
- OV2640 camera supports QVGA (320x240) with PSRAM
- Unit cost approximately USD 8-10, viable for fleet-wide deployment

The NEO-6M GPS module was chosen for:

- Positional accuracy of approximately 2.5 meters under open sky
- Reliable NMEA sentence output (GGA, RMC)
- UART communication at 9600 baud, compatible with ESP32
- Low power consumption (approximately 45 mA during acquisition)

The functional division between the two components is straightforward: the NEO-6M continuously outputs GGA and RMC NMEA sentences containing latitude, longitude, speed, and fix status, while the ESP32-CAM captures JPEG images at QVGA resolution (320x240) when triggered by the telemetry timer. Both data streams are combined into a single multipart HTTP POST request and transmitted to the backend every 60 seconds.



Hardware Block Diagram

Figure 4.1: Hardware block diagram showing the ESP32-CAM, NEO-6M GPS module, power supply, and their interconnections.

4.2.2 Hardware Assembly and Wiring

The physical assembly follows a specific placement strategy optimized for each component’s function:

- **ESP32-CAM** — Mounted on the bus ceiling, facing downward at approximately 45 degrees, to capture a top-down view of the passenger area. This angle maximizes visible floor area and minimizes occlusion from seat backs.
- **NEO-6M GPS** — Positioned near a window or on the dashboard to ensure consistent satellite signal reception.
- **Power** — Drawn from the bus 12V electrical system through a buck converter stepping down to 5V. A 1000 uF capacitor across the GPS power input smooths voltage fluctuations from the alternator and starter motor.

The wiring connections between the ESP32-CAM and NEO-6M are minimal:

Signal	ESP32-CAM Pin	NEO-6M Pin	Wire Color
GPS Data Out	GPIO 14 (UART2 RX)	TX	Yellow
GPS Commands	GPIO 15 (UART2 TX)	RX	White
Power	5V	VCC	Red
Ground	GND	GND	Black

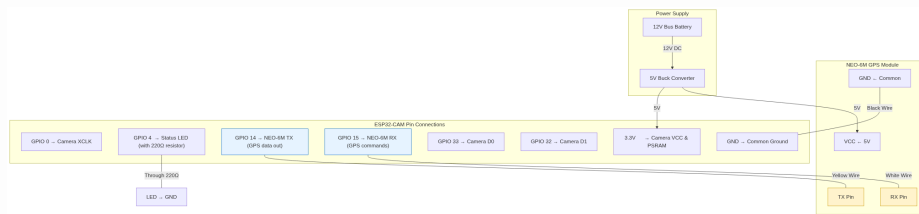
Table 4.2: Wiring connections between ESP32-CAM and NEO-6M GPS module.

These GPIO pins were specifically chosen because they do not conflict with the camera data pins (GPIO 0-5, 12-16 for camera data and clock signals).

Environmental considerations during assembly included:

- Vibration damping using rubber grommets for mounting screws
- Heat management ensuring adequate airflow around the ESP32 voltage regulator

- Antenna orientation positioning the GPS antenna with a clear view of the sky



Wiring Diagram

Figure 4.2: Wiring schematic showing connections between ESP32-CAM, NEO-6M GPS, and power supply.

4.2.3 Firmware Implementation

The firmware follows a structured architecture with a setup phase and a continuous main loop. During setup:

1. Camera is initialized with PSRAM detection (falling back to QOVGA if PSRAM unavailable)
2. GPS UART is configured at 9600 baud on GPIO 14/15
3. Wi-Fi connection is established with automatic reconnection logic
4. A startup heartbeat is sent to the backend /health endpoint

The main loop operates on a non-blocking design:

- Continuously feeds GPS data from the UART buffer
- Checks a 15-second heartbeat timer
- Checks a 60-second telemetry timer
- Parses NMEA sentences using TinyGPSPlus library

A three-tier GPS resolution strategy ensures reliable position data:

1. **Tier 1** — Fresh GPS fix with valid checksum
2. **Tier 2** — Last known good coordinate from history
3. **Tier 3** — Configurable fallback coordinate (default: Addis Ababa city center at 9.032, 38.752)

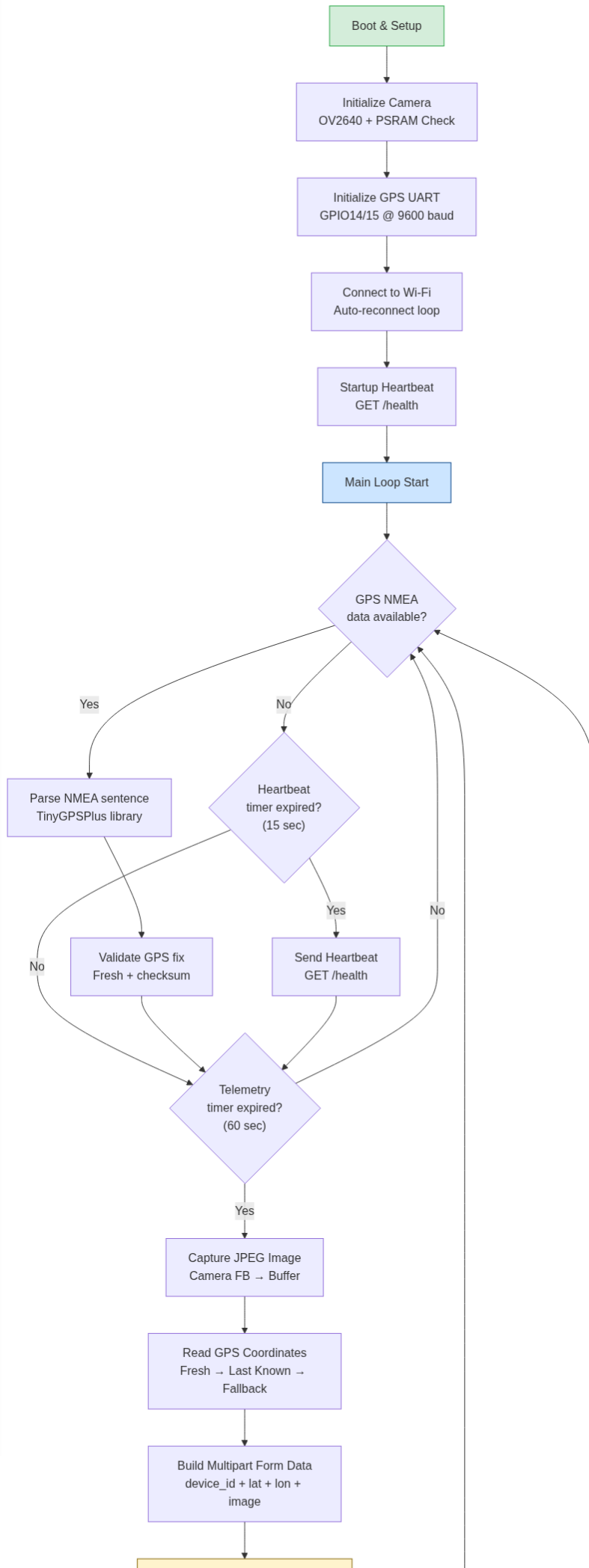
When the telemetry timer expires, the firmware:

1. Captures a JPEG frame from the camera
2. Reads current GPS coordinates

3. Constructs multipart form-data payload (device_id, plate, bus_type, lat, lon, speed, capacity, image)
4. Transmits via HTTPS POST to /api/v1/gateway/esp32/telemetry
5. Blinks status LED (GPIO 4) — green for success, red for failure

Reliability features include:

- Watchdog timer that resets the ESP32 if the main loop hangs during large uploads
- Camera self-check and recovery on initialization failure
- Telemetry counter tracking attempts versus successful transmissions
- Built-in diagnostic web server at the ESP32 IP address showing GPS fix status, NMEA history, Wi-Fi signal strength, and telemetry success rate



Firmware Flowchart

Figure 4.3: Complete firmware execution flow from boot through the main telemetry loop.

4.3 Software Implementation

The software subsystem encompasses the backend API server, two Next.js frontend dashboards, and the Flutter mobile application. This section describes the development environment, module organization, database schema, and user interface implementation.

4.3.1 Development Environment Setup

The development environment was configured as follows:

Backend:

- Python 3.11 with FastAPI as the web framework
- SQLAlchemy 2.0 with async support for database operations
- Alembic for schema migrations
- PostgreSQL 15 with PostGIS extension for geospatial queries
- Redis 7 for caching and pub/sub messaging
- Docker Compose orchestrating three containers: FastAPI (port 8000), PostgreSQL (port 5433), Redis (port 6479)

Frontend (Admin Dashboard):

- Node.js 20 with Next.js 14 (App Router)
- TypeScript 5, Tailwind CSS 3.4
- shadcn/ui component library built on Radix UI primitives
- Leaflet for map rendering, Recharts for data visualization

Frontend (Driver Dashboard):

- Same stack as admin dashboard
- Separate Next.js project running on port 3001
- Optimized for tablet use inside the bus

Mobile Application:

- Flutter 3.x with Dart 3

- Riverpod for state management, GoRouter for declarative navigation
- Dio for HTTP communication
- Firebase Cloud Messaging (FCM) for push notifications
- Supports both Android and iOS from a single codebase

Environment variables managed through `.env` files for each project:

Variable	Purpose
SECRET_KEY	JWT token signing
DATABASE_URL	PostgreSQL connection string
REDIS_URL	Redis connection string
GOOGLE_CLIENT_ID	Google OAuth client ID
FCM_SERVER_KEY	Firestore push notification key
USE_ML_FOR_PROD	Toggle ML-based ETA prediction

Table 4.3: Critical environment variables for system configuration.

4.3.2 Module Implementation

Backend Modules. The backend is organized into 21 API router modules, each handling a specific domain:

Router	Endpoints	Purpose
auth	13	Registration, login (email/password, OAuth), driver login, email verification, password reset
vehicles	7	Vehicle registry, position tracking, route assignment, telemetry ingestion
routes / stops	6	Route and stop CRUD with sequence ordering
assignments	3	

Router	Endpoints	Purpose
		Admin trip management (start/end/list active)
driver_assignments	3	Driver self-service trip management
gateway	1	ESP32 multipart telemetry upload
tracking	1	SIM7600 GPS telemetry
search	2	Point-to-point journey planning with live ETAs
admin_dashboard	11	Dashboard charts, ML training, ETA preview, settings
admin_users	7	User CRUD, search, role management
crowd	1	CV crowd density results
favorites	3	User saved routes
ratings	3	Journey feedback
notifications	3	FCM token registration, proximity alerts
pairing	3	Device pairing codes, verification, unpairing
trip_history	2	Trip history read endpoints
users	2	Profile management
websocket	1	Admin live stream
websocket_mobile	1	Passenger mobile stream
admin	1	ML toggle

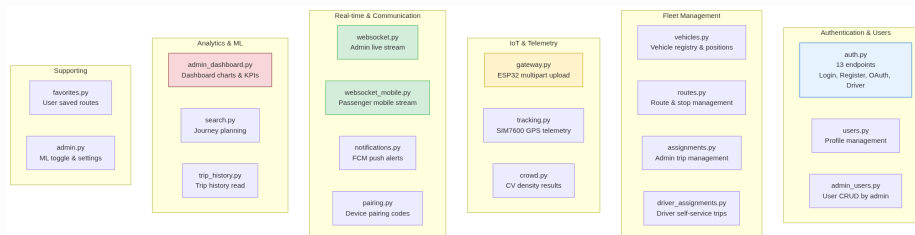
Table 4.4: API router modules with endpoint counts and purposes.

The middleware stack processes incoming requests in a specific order (outermost to innermost):

1. **CORS** — Configurable origins, credentials handling
2. **Security Headers** — HSTS (1 year), CSP, X-Frame-Options DENY, no-cache for API
3. **Request Validation** — Body size limits (1 MB JSON, 10 MB multipart), content-type enforcement, path traversal detection
4. **Firewall** — IP blocklist, auto-ban on abuse, burst detection, user-agent blocking

Rate limiting uses a dual system:

- Slowapi decorators for per-endpoint limits (e.g., 20/min for auth, 300/min for telemetry)
- Redis-backed sliding window for IP-based throttling with tiered limits



API Router Organization

Figure 4.4: API router organization showing 21 routers grouped by functional domain.

Frontend Modules. The admin dashboard follows a component-based architecture:

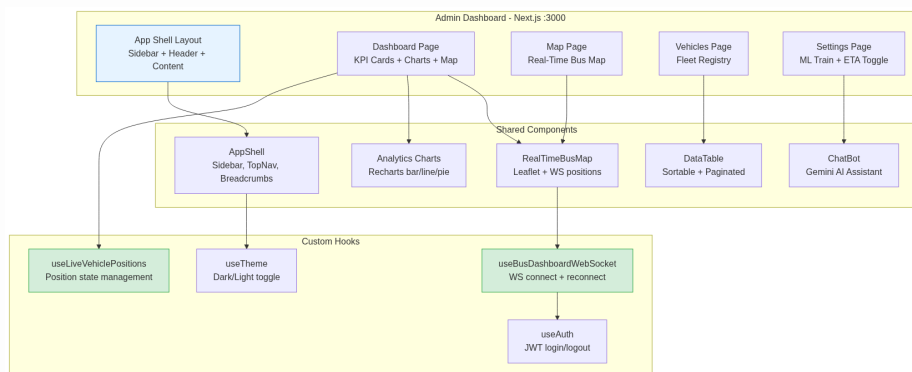
- **App Shell** — Collapsible sidebar, top navigation bar, breadcrumb trail
- **Dashboard Page** — KPI cards (active assignments, vehicles, routes, users), ETA accuracy charts, live map
- **Vehicles Page** — Fleet registry with pairing code generation
- **Routes Page** — Route and stop management
- **Assignments Page** — Live trip monitoring
- **Users Page** — Driver and admin account management
- **Settings Page** — ML model training, ETA mode toggle, data cleanup

Custom hooks encapsulate complex logic:

- `useBusDashboardWebSocket` — WebSocket connections with auto-reconnect and message filtering by vehicle ID
- `useAuth` — JWT-based authentication with automatic 401 redirect
- `useLiveVehiclePositions` — Merges WebSocket position updates with REST-fetched initial data
- `useTheme` — Dark/light theme switching

The bus driver dashboard authentication flow has four steps:

1. **Device Pairing** — First-time setup with a pairing code
2. **Device Unlock** — IMEI and dashboard password
3. **Driver Login** — Username and password
4. **Main Dashboard** — Live map, speed, crowd density, route progress, start/end ride buttons



Component Hierarchy

Figure 4.5: Admin dashboard component hierarchy showing the App Shell, page components, shared hooks, and UI components.

Mobile Application Modules. The Flutter application follows a feature-based clean architecture:

- **Core Layer** — `ApiClient` (Dio with auth interceptor), `AuthInterceptor` (JWT attachment and 401 handling), `SessionEventBus` (cross-feature session state), `FCMService` (push notification registration), `AppRouter` (GoRouter with auth redirects)
- **Auth Feature** — Login, register, password reset, email verification
- **Home Feature** — Main map view with live bus positions
- **Search Feature** — Journey planning with stop picker, results, route detail
- **Favorites Feature** — Saved routes and stops

- **Ratings Feature** — Journey feedback
- **Notifications Feature** — Proximity alert configuration

The application uses a dual transport strategy:

- **Primary** — WebSocket for real-time updates (sub-second latency)
- **Fallback** — REST polling every 15 seconds when WebSocket disconnected
- **Reconnection** — Exponential backoff from 1 second to 30 seconds cap

4.3.3 Database and API Implementation

The database schema consists of 14 tables designed to support the full range of system operations:

Table	Purpose	Key Columns
users	Account information with role-based access	id, username, email, password_hash, role (passenger/driver/admin)
vehicles	Bus registry with IoT device binding	id, plate_number, device_id (IMEI), capacity, route_id (FK)
routes	Route definitions	id, route_number, direction (forward/reverse), name, origin, destination
stops	Bus stop metadata	id, name, lat, lon, base_dwell_time, is_terminal, peak_multiplier
route_stops	Stop sequence per route (many-to-many)	route_id (FK), stop_id (FK), sequence_order
assignments	Active trips (driver + vehicle + route)	id, driver_id (FK), vehicle_id (FK), route_id (FK),

Table	Purpose	Key Columns
		start_time, end_time, status
driver_bus_sessions	Driver logins on bus devices	id, driver_id (FK), vehicle_id (FK), login_at, logout_at, status
raw_telemetry	Bronze layer unprocessed hardware data	id, vehicle_id (FK), raw_lat, raw_lon, raw_payload (JSONB)
trip_history	Stop-level events for ML training	id, assignment_id (FK), stop_id (FK), arrival_time, dwell_time, occupancy_level, heuristic_eta, ml_eta, actual_travel_time
model_performance	Heuristic vs ML comparison	id, trip_history_id (FK), heuristic_error, ml_error
favorites	User saved routes	id, user_id (FK), route_id (FK), nickname
ratings	Journey feedback	id, user_id (FK), assignment_id (FK), score (1-5), comment
notification_settings	Proximity alerts	id, user_id (FK), route_id (FK), stop_id (FK), lead_time_minutes
system_settings	Runtime configuration	id, key, value

Table 4.5: Database schema showing all 14 tables with purposes and key columns.

Key design decisions:

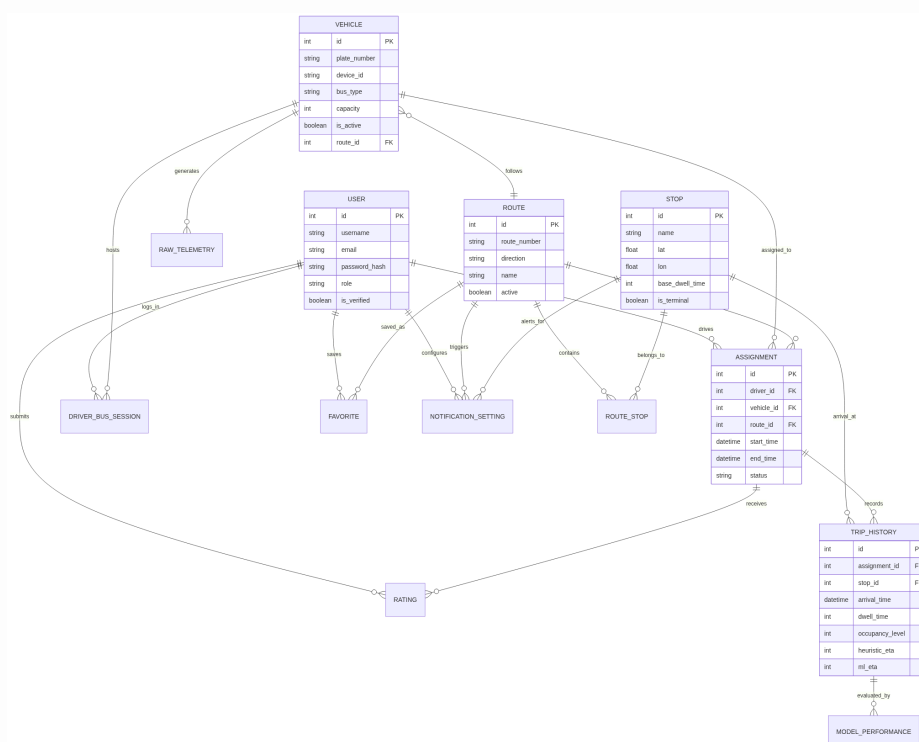
- Async SQLAlchemy with AsyncSession for non-blocking database operations
- `server_default=func.now()` for automatic timestamp management
- Composite primary keys for `route_stops` (`route_id`, `stop_id`)
- Foreign key cascading deletes where appropriate
- JSONB column in `raw_telemetry` for flexible payload storage

Redis serves as the real-time data layer with these key patterns:

Key Pattern	Type	TTL	Purpose
<code>bus:live:{plate}</code>	Hash	5 min	Live position (lat, lon, speed, occupancy, assignment_id)
<code>veh:cv:{plate}</code>	Hash	5 min	CV results (people_count, crowd_density, confidence, method)
<code>veh:pos:{plate}</code>	String (JSON)	5 min	Last known position
<code>veh:hist:{plate}</code>	List	5 min	Last 5 GPS coordinates for validation
<code>route:{no}:stop:{id}</code>	Hash	5 min	Per-stop ETA data (eta_seconds, distance_m, occupancy, mode)
<code>active_buses</code>	Geo	—	Geospatial index for nearby bus lookup
<code>pipe:positions</code>	Stream	—	Live position stream for consumers

Key Pattern	Type	TTL	Purpose
ws:vehicle_position	Pub/Sub	—	Cross-worker position broadcast channel
ws:cv_result	Pub/Sub	—	Cross-worker CV result broadcast channel

Table 4.6: Redis key patterns with data types, TTLs, and purposes.



ER Diagram

Figure 4.6: Entity-relationship diagram showing all 14 database tables and their relationships.

The API follows RESTful conventions with an `/api/v1` prefix. All endpoints use Pydantic schemas for request validation and response serialization. The authentication system uses:

- **JWT tokens** — HS256 algorithm, 24-hour expiry, payload: `{sub: user_id, exp: timestamp, type: "access"}`
- **Role-based access** — `RequireAdmin` (admin only), `RequireDriver` (driver or admin), `CurrentUser` (any authenticated)

- **Public endpoints** — Vehicle positions, route ETAs, health check require no authentication

4.3.4 User Interface Implementation

Admin Dashboard UI:

- Dark-first design with pure black background (#09090B), neon cyan and green accents
- Space Grotesk typeface for headings, Inter for body text
- Sidebar navigation with collapsible sections for fleet management, operations, and analytics
- KPI cards showing: active assignments, total vehicles, routes, users
- Charts: assignments over time (bar), occupancy distribution (pie), ETA accuracy (line), route usage (bar)
- Real-time bus map with Leaflet: custom markers showing plate number, color-coded by occupancy (green/yellow/red)
- Bus detail popup: current speed, driver name, route, crowd density

Bus Driver Dashboard UI:

- Driver-first design with large touch targets (minimum 48px height)
- Minimal text, glanceable information for use while driving
- Live map with bus position and route overlay
- Stat cards: speed, crowd density, next stop ETA
- Route progress bar showing passed stops (checkmark), current stop (dot), upcoming stops (circle)
- Prominent start/end ride buttons
- Glove-compatible design, works in varying lighting conditions

Mobile App UI:

- Material 3 design with custom theming
- Main map screen: user location, nearby stops, live bus positions with occupancy badges
- Journey search: autocomplete stop picker, results with ETA, crowd level, walking distance
- Bottom navigation: Map | Search | Favorites | Profile
- Push notifications for proximity alerts

4.4 AI/ML Model Implementation

The system implements two machine learning pipelines: a computer vision pipeline for crowd density detection and a regression model for ETA prediction. Both pipelines were designed to work with limited computational resources and to degrade gracefully when ML models are unavailable.

4.4.1 Crowd Density Detection Pipeline

The crowd density detection system addresses the challenge of estimating passenger count from a ceiling-mounted camera in a moving bus. This is fundamentally different from street-level pedestrian detection because:

- The camera angle is top-down rather than side-on
- Passengers are often partially occluded by seats and luggage
- Lighting conditions vary dramatically between day and night
- Motion blur from bus movement affects image quality

The system uses a three-tier detection architecture:

Tier 1 — YOLOv8-nano Full-Body Detection:

- Model: YOLOv8-nano (approximately 6 MB), auto-downloaded from Ultralytics
- Detects COCO class 0 (person) at confidence threshold 0.35
- Catches upright, fully visible passengers
- Runs on best available device (CUDA > MPS > CPU)

Tier 2 — YOLOv8-nano Face Detection:

- Separate face detection model at confidence threshold 0.30
- Catches passengers whose bodies are occluded but faces visible
- Gracefully degrades if face model unavailable

Tier 3 — Head-Blob Contour Analysis:

- Classical CV pipeline for top-down camera angles
- Steps: grayscale conversion, CLAHE contrast enhancement, adaptive thresholding, morphological cleanup

- Contour filtering: area 800-25000 px, circularity ≥ 0.45 , aspect ratio 0.5-2.0, solidity ≥ 0.5
- Parameters determined empirically from sample bus interior images

Deduplication:

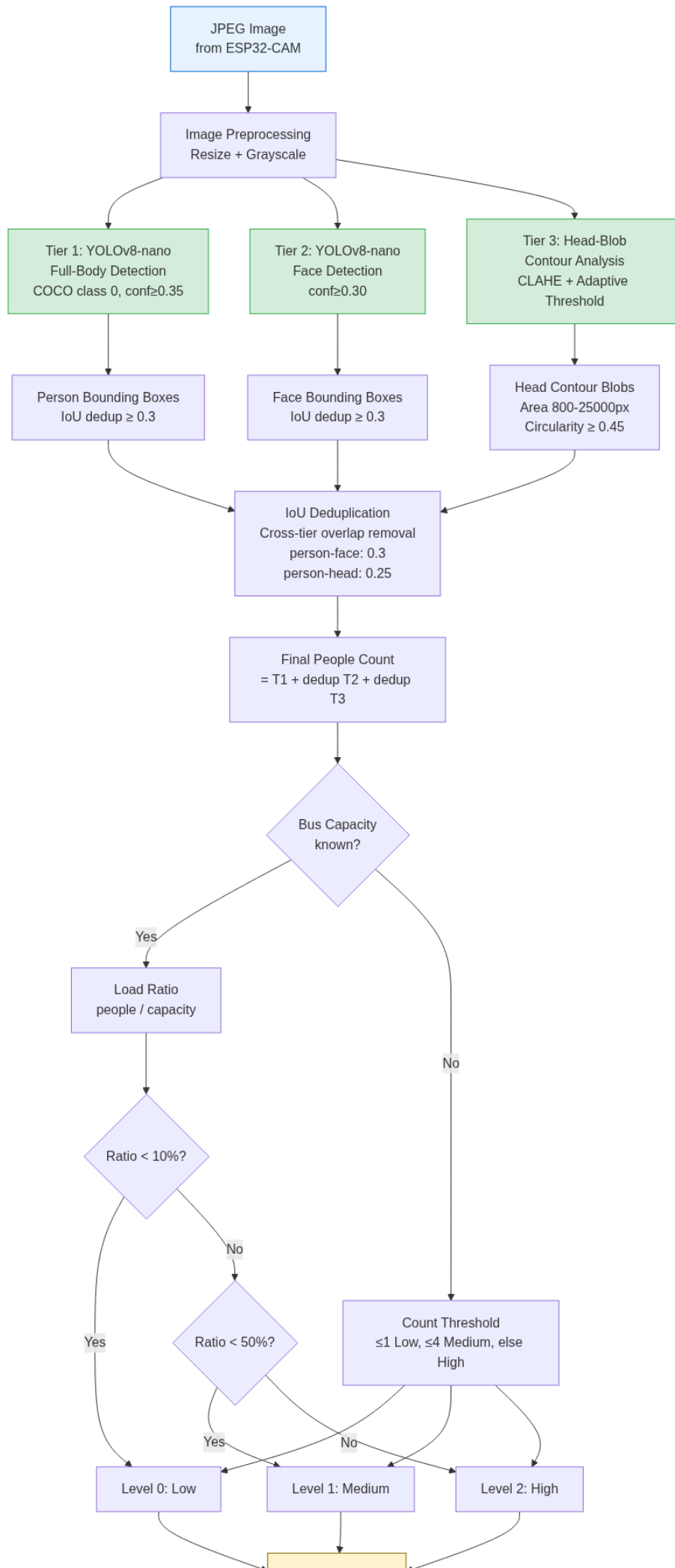
- IoU-based overlap removal prevents double-counting
- Person-face overlap threshold: 0.30
- Person-head and face-head overlap threshold: 0.25
- Final count = unique Tier 1 + unique Tier 2 + unique Tier 3

Density Classification:

Condition	Low (0)	Medium (1)	High (2)
With capacity	load ratio < 10%	load ratio < 50%	load ratio $\geq$ 50%
Without capacity	count $\leq$ 1	count $\leq$ 4	count $\geq$ 5

Table 4.7: Density classification thresholds with and without known bus capacity.

Fallback: If YOLO models fail to load, the system falls back to OpenCV HOG person detector combined with foreground ratio analysis.



CV Pipeline Flowchart

Figure 4.7: Computer vision pipeline showing the three detection tiers, deduplication, and density classification.

4.4.2 ETA Prediction Model

The ETA prediction system uses a two-tier approach:

Heuristic Baseline:

- Haversine formula for great-circle distance between bus position and each remaining stop
- Average speed: 10 m/s (36 km/h, representative of urban bus speeds in Addis Ababa)
- Dwell time = sum of (base_dwell_time * peak_multiplier * occupancy_factor) for each remaining stop
- Peak hours: 7:00-9:30 (1.5x), 16:30-19:30 (1.8x)
- Occupancy penalty: Level 0 = 1.0x, Level 1 = 1.2x, Level 2 = 1.4x

ML Residual Correction:

- Model: RandomForestRegressor (120 estimators, max_depth=14)
- Predicts the residual error of the heuristic calculation (not absolute ETA)
- 12 input features:

#	Feature	Description
1	route_id	Route identifier
2	stop_id	Stop identifier
3	stop_sequence	Position in route stop sequence
4	remaining_stops	Number of stops remaining
5	distance_m	Distance to stop in meters
6	base_dwell_time	Base dwell time at stop
7	peak_multiplier	Peak hour multiplier
8	hour	Hour of day (0-23)
9	day_of_week	Day of week (0-6)
10	is_peak_hour	Boolean peak hour flag
11	occupancy_level	Current occupancy (0/1/2)

#	Feature	Description
12	heuristic_eta	Heuristic ETA value

Table 4.8: Twelve input features for the RandomForest ETA residual model.

Training Pipeline:

- Data source: trip_history table with actual travel times between consecutive stops
- Target variable: actual_travel_time - heuristic_eta (the residual)
- Minimum 50 samples required before training
- 80/20 train/test split
- Evaluation metrics: MAE (Mean Absolute Error), RMSE (Root Mean Square Error)
- Model serialized to delay_predictor.joblib with feature names

Inference:

- System checks USE_ML_FOR_PROD toggle in database
- If enabled and model loaded: final_eta = heuristic_eta + predicted_residual
- If disabled or model unavailable: returns heuristic ETA with mode “heuristic”
- Graceful degradation ensures ETA estimates are always available

Current Bus Position
lat, lon from GPS

Find Nearest Stop
on assigned route

Compute Haversine
Distance
to each remaining stop

Heuristic ETA Calculation
 $\text{travel_time} = \text{distance} / 10 \text{ m/s}$
 $\text{dwell} = \text{stops} \times \text{base_dwell} \times \text{peak_mult}$

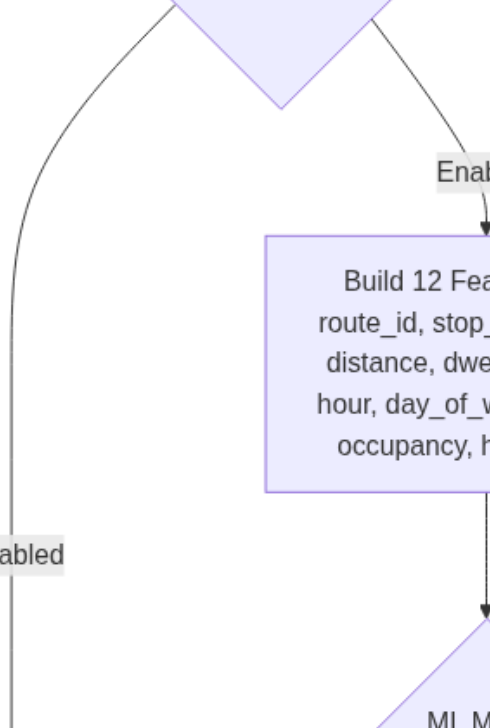
Admin Toggle
USE_ML_FOR_PROD?

Enabled

Build 12 Feature Vector
route_id, stop_id, sequence
distance, dwell, peak_mult
hour, day_of_week, is_peak
occupancy, heuristic_eta

Disabled

ML Model



ETA Computation Flowchart

Figure 4.8: ETA computation flow showing heuristic calculation, ML residual prediction, and the admin toggle.

4.5 System Integration

System integration connects the hardware firmware, backend processing, and client applications into a cohesive real-time tracking system.

4.5.1 Hardware-Software Integration

The integration between the ESP32-CAM firmware and the backend follows a well-defined protocol:

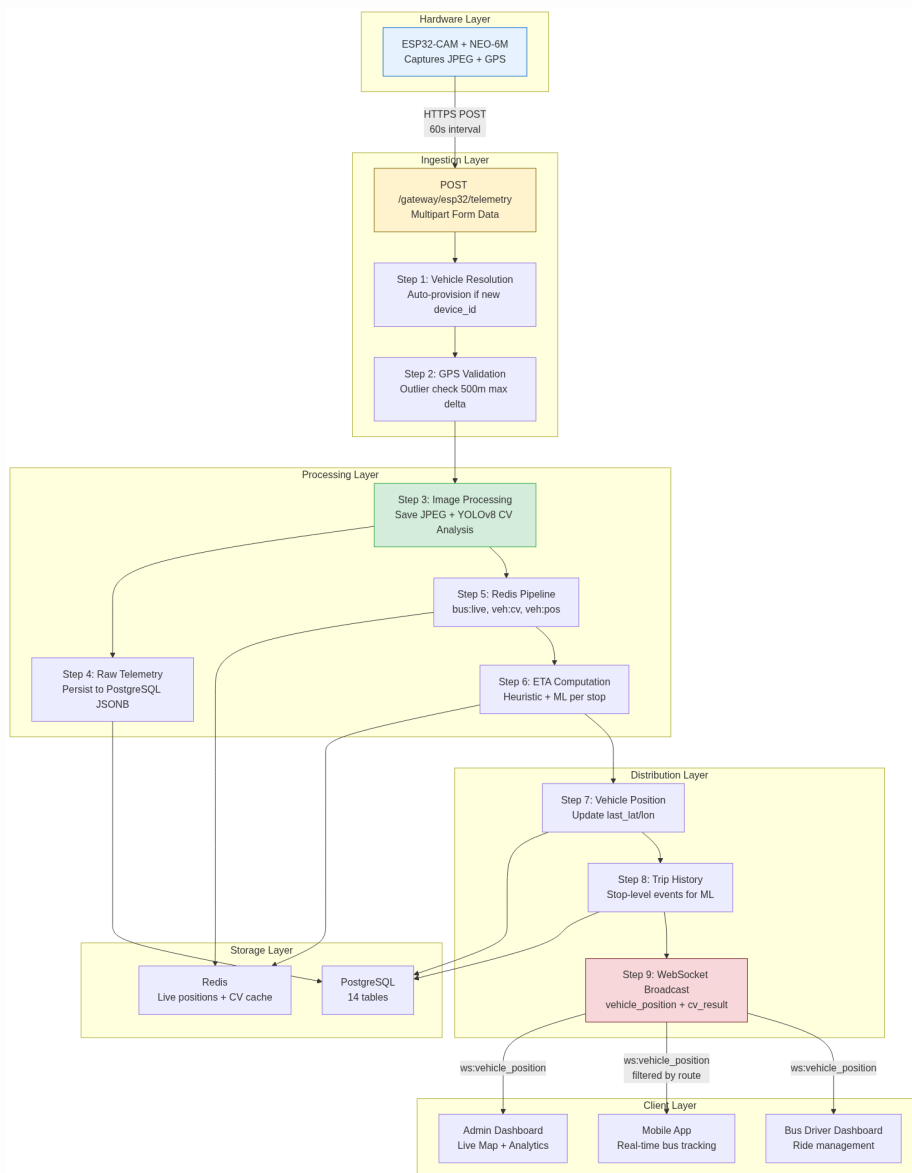
1. ESP32 boots, reads stored Wi-Fi credentials and device configuration
2. Connects to local Wi-Fi network
3. Sends startup heartbeat to /health endpoint
4. Every 60 seconds: captures JPEG + reads GPS, constructs multipart payload, sends HTTPS POST to /api/v1/gateway/esp32/telemetry

The backend gateway endpoint delegates to the unified `process_telemetry()` pipeline:

Step	Action	Output
1	Vehicle resolution by device_id	Auto-provision if new device
2	GPS validation against last 5 positions	Reject outliers > 500m delta
3	Image storage + CV analysis	Crowd density (0/1/2)
4	Raw telemetry persistence	JSONB record in PostgreSQL
5	Redis live pipeline update	bus:live, veh:cv, veh:pos
6		

Step	Action	Output
	ETA computation per stop	Heuristic + optional ML
7	Vehicle position update	last_lat, last_lon in PostgreSQL
8	Trip history recording	Stop-level events for ML
9	WebSocket broadcast	vehicle_position + cv_result

Table 4.9: Nine-step telemetry processing pipeline from ingestion to broadcast.



End-to-End Data Flow

Figure 4.9: End-to-end data flow from ESP32 through all nine processing steps to the client applications.

4.5.2 Communication Interface Implementation

REST API:

- 80+ endpoints under `/api/v1/` prefix
- Pydantic schemas for request validation and response serialization
- Async SQLAlchemy throughout for non-blocking database operations
- Consistent error responses with HTTP status codes

WebSocket with Redis Pub/Sub:

The critical architectural fix for multi-worker deployment:

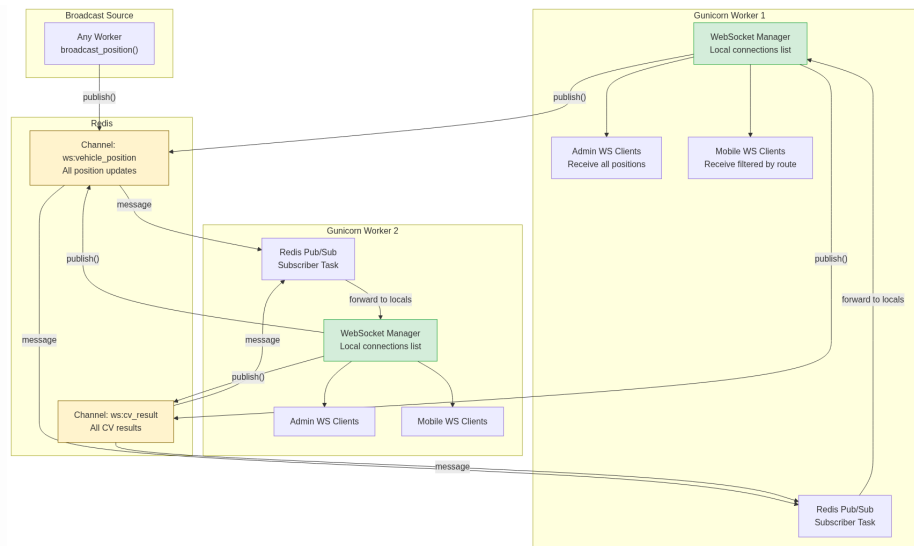
- **Problem:** In-memory connection list broke with multiple Gunicorn workers (each worker had its own list)
- **Solution:** Redis Pub/Sub fan-out — any worker publishes to `ws:vehicle_position` channel, all workers receive and forward to local connections
- **Admin WebSocket** — Receives all vehicle positions (no filter)
- **Mobile WebSocket** — Includes `subscribed_route_id`, receives only positions for subscribed route

Mobile App Dual Transport:

- WebSocket primary (sub-second latency)
- REST polling fallback (every 15 seconds)
- Exponential backoff reconnection (1s to 30s cap)
- Route subscription via WebSocket messages

FCM Push Notifications:

- Background worker checks every 60 seconds for buses approaching user-subscribed stops
- 5-minute cooldown per notification to prevent spam
- Notification includes bus plate, route, ETA, and crowd level



WebSocket Pub/Sub Architecture

Figure 4.10: WebSocket Redis pub/sub architecture showing two Gunicorn workers with local connections connected via Redis pub/sub channels.

4.5.3 Integration Testing

Integration testing verified interoperability between all subsystems:

- **ESP32-to-backend:** curl with multipart form data simulates device uploads without physical hardware
- **WebSocket verification:** Connect to /api/v1/ws/live with admin JWT, verify position messages arrive after telemetry posts
- **End-to-end scenario:** Register device -> send telemetry -> verify vehicle on admin live map -> verify CV results in Redis -> verify ETA computation -> verify mobile app receives updates
- **Simulation mode:** Python scripts generate synthetic GPS data along predefined routes for testing without physical hardware
- **Concurrent devices:** Multiple simultaneous device uploads verify multi-worker WebSocket broadcasting

4.6 Deployment and Configuration

The production deployment uses:

- **Application server:** Gunicorn with 2-4 Uvicorn workers (depending on CPU count)

- **Container:** Docker multi-stage build (builder + runtime), final image approximately 350 MB
- **Database:** Supabase (PostgreSQL with PostGIS)
- **Cache:** Upstash (Redis with TLS)
- **Frontend:** Next.js standalone output, served via Nginx reverse proxy
- **Mobile:** Google Play Store and Apple App Store distribution

Database migrations are managed through Alembic (`alembic upgrade head`). Data retention policies:

Data Type	Retention Period	Cleanup Trigger
Raw telemetry	30 days	Manual via <code>POST /admin/cleanup</code>
Trip history	365 days	Manual via <code>POST /admin/cleanup</code>
Redis live data	5 minutes	Automatic TTL expiration
FCM tokens	30 days	Automatic TTL expiration

Table 4.10: Data retention policies for different data types.

4.7 Challenges and Solutions

Multi-worker WebSocket Broadcasting. The initial in-memory connection list broke with multiple Gunicorn workers. Each worker maintained its own WebSocket connections, so position updates from telemetry processed by Worker 1 would not reach clients connected to Worker 2. The solution introduced Redis Pub/Sub as a fan-out layer. When any worker broadcasts a position update, it publishes to a Redis channel that all workers subscribe to. Each worker then forwards the message to its local connections. This pattern scales linearly with worker count.

GPS Data Quality. The ESP32 in a moving bus experiences GPS signal loss in urban canyons, tunnels, and dense tree cover. Raw GPS data occasionally contains outliers with errors exceeding 100 meters. The solution implements a three-tier GPS resolution strategy: use a fresh fix with valid checksum, fall back to the last known good coordinate, and

finally use a configurable fallback coordinate. The GPS validation step in the telemetry pipeline rejects coordinates that deviate more than 500 meters from the last five known positions.

Crowd Detection Accuracy. Standard pedestrian detection models trained on street-level images perform poorly on the top-down interior bus camera angle. Passengers appear as head-and-shoulder blobs rather than full-body figures, and occlusion from seats and luggage is severe. The three-tier detection architecture addresses this by combining full-body detection (for standing passengers), face detection (for seated passengers with visible faces), and head-blob contour analysis (for top-down angles). IoU-based deduplication prevents double-counting across tiers.

ML Model Cold Start. The RandomForest ETA model requires at least 50 trip history samples before it can be trained. During the initial deployment period, no training data exists. The system handles this gracefully by always providing heuristic ETA estimates, with the ML correction applied only when sufficient data has been collected and the admin enables the ML toggle. This ensures the system is useful from day one, with accuracy improving over time.

Cross-Origin Frontend Separation. The admin dashboard (port 3000) and bus driver dashboard (port 3001) are separate Next.js applications with different authentication requirements. The CORS configuration on the backend allows both origins while maintaining security through JWT validation. The shared JWT authentication system ensures that a driver logged into the bus dashboard can also access driver-specific admin endpoints when needed.

ESP32 Memory Constraints. The ESP32 has only 520 KB of SRAM, which must accommodate the Wi-Fi stack, GPS parsing, camera driver, and HTTP client simultaneously. JPEG capture at QVGA resolution requires approximately 300 KB for the frame buffer. The solution uses external PSRAM (4 MB on most ESP32-CAM modules) for the frame buffer, leaving the internal SRAM for program execution. Without PSRAM, the firmware falls back to QQVGA resolution (160x120), which fits in internal SRAM but provides lower image quality for crowd detection.

Appendix A: Core Code Snippets

This appendix presents selected code snippets that illustrate key implementation decisions discussed in Chapter 4.

A.1 Driver Assignment Endpoint

```
# backend/app/api/v1/driver_assignments.py
@router.post("/driver/assignments/start",
             response_model=DriverAssignmentOut)
async def start_driver_assignment(
    body: DriverAssignmentStartBody,
    current_user: RequireDriver,
    db: AsyncSession = Depends(get_db),
):
    session = await
        crud_driver_session.get_active_session_for_driver(
            db, current_user.id
        )
    if not session:
        raise HTTPException(404, "No active driver session")
    if not await crud_route.get_route_by_id(db, body.route_id):
        raise HTTPException(404, "Route not found")
    existing = await
        crud_assignment.get_active_assignment_by_vehicle(
            db, session.vehicle_id
        )
    if existing:
        raise HTTPException(409, "Vehicle already has an active
            assignment")
    a = await crud_assignment.create_assignment(
        db, current_user.id, session.vehicle_id, body.route_id
    )
    return _to_out(a)
```

Listing A.1: Driver-scoped assignment start endpoint using RequireDriver auth.

A.2 ESP32 GPS Fallback Logic

```
bool selectGpsFix(float &lat, float &lon, float &spd) {
    if (gps.location.isValid() && gps.location.isUpdated()) {
```

```

        lat = gps.location.lat();
        lon = gps.location.lng();
        spd = gps.speed.kmph();
        return true;
    }
    if (lastLat != 0.0 && lastLon != 0.0) {
        lat = lastLat; lon = lastLon; spd = 0.0;
        return true;
    }
    lat = 9.032; lon = 38.752; spd = 0.0;
    return false;
}

```

Listing A.2: Three-tier GPS resolution strategy in ESP32 firmware.

A.3 WebSocket Redis Pub/Sub

```

class ConnectionManager:
    async def broadcast_position(self, vehicle_id: int, data:
        dict):
        redis = await get_redis()
        await redis.publish("ws:vehicle_position",
            json.dumps(data))

    async def _subscribe_loop(self):
        redis = await get_redis()
        pubsub = redis.pubsub()
        await pubsub.subscribe("ws:vehicle_position",
            "ws:cv_result")
        async for message in pubsub.listen():
            if message["type"] == "message":
                await
                self._forward_to_locals(json.loads(message["data"]))

```

Listing A.3: WebSocket manager with Redis pub/sub for multi-worker broadcast.

A.4 YOLOv8 Crowd Density Detection

```

class YoloDetector:
    def detect(self, image_path: str) -> DetectionResult:
        img = cv2.imread(image_path)
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

```

```

person_boxes = self.person_model(img, conf=0.35)[0].boxes
face_boxes = self.face_model(img, conf=0.30)[0].boxes
head_blobs = self._detect_head_blobs(gray)
unique_faces =
self._deduplicate(person_boxes, face_boxes, 0.3)
unique_heads = self._deduplicate_all(person_boxes,
head_blobs, 0.25)
total = len(person_boxes) + len(unique_faces) +
len(unique_heads)
return DetectionResult(people_count=total,
crowd_density=self._classify(total))

```

Listing A.4: Three-tier crowd density detection with IoU deduplication.

A.5 Haversine Heuristic ETA

```

def calculate_eta_heuristic(bus_lat, bus_lon, stops, nearest_idx,
occupancy=0):
    results = {}
    peak_mult = 1.5 if (7 <= hour <= 9.5) else (1.8 if (16.5 <=
hour <= 19.5) else 1.0)
    occ_factor = 1.0 if occupancy == 0 else (1.2 if occupancy ==
1 else 1.4)
    for i in range(nearest_idx, len(stops)):
        distance = haversine(bus_lat, bus_lon, stops[i].lat,
stops[i].lon)
        travel_time = distance / 10.0
        dwell = sum(s.base_dwell_time * peak_mult * occ_factor
for s in stops[nearest_idx:i+1])
        results[stops[i].id] = ETAResult(eta_seconds=travel_time
+ dwell, distance_m=distance)
    return results

```

Listing A.5: Heuristic ETA calculation with peak-hour and occupancy adjustments.